

Entregables T2 — Evolución temporal (Sebastián)

ítems 3 y 4: integración directa (RK4) y acción del exponencial (Krylov)

Sebastián

3 de junio de 2026

Documento de revisión autocontenido. Los mismos fragmentos se \input en el documento maestro mpemba_cuantico.tex.

Índice

1 Evolución temporal (a): integración directa con RK4	1
1.1 Explicación del framework	1
1.2 Del modelo teórico a la discretización (paso a paso)	1
1.3 Implementación serial y líneas de código	2
1.4 La directiva de paralelización (OpenMP)	2
1.5 Justificación de la paralelización: OpenMP	2
1.6 Comparación serial vs paralelo	3
1.7 Resultados y validación	3
2 Evolución temporal (b): acción del exponencial por Krylov	5
2.1 Explicación del framework	5
2.2 Del modelo teórico a la discretización (paso a paso)	5
2.3 Implementación serial y líneas de código	6
2.4 La directiva de paralelización (OpenMP)	6
2.5 Justificación de la paralelización: OpenMP	6
2.6 Comparación serial vs paralelo	7
2.7 Comparación de método (a vs b): el resultado central	7
2.8 Resultados y conclusión del método (b)	8

1 Evolución temporal (a): integración directa con RK4

1.1 Explicación del framework

La tarea de *evolución temporal* consiste en obtener la trayectoria del operador densidad $\rho(t)$ que resuelve la ecuación maestra de Lindblad $\dot{\rho} = \mathcal{L}[\rho]$ (ec. (??)) a partir de una preparación ρ_0 , para trazar las curvas de distancia al equilibrio $\mathcal{D}(\rho_t || \rho_{ss})$ que revelan (o descartan) el efecto Mpemba. El método (a) es la vía **directa**: integrar la ecuación diferencial paso a paso, sin construir el superoperador $\mathbb{L}$ de dimensión $d^2 \times d^2$, aplicando en su lugar la acción *matrix-free* $\mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} (L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\})$.

1.2 Del modelo teórico a la discretización (paso a paso)

Paso 0 — Modelo teórico. El punto de partida es la ecuación maestra GKSL (??) para la cadena de Ising transversa disipativa:

$$\dot{\rho} = \mathcal{L}[\rho] = -i[H, \rho] + \sum_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{L_{\mu}^{\dagger} L_{\mu}, \rho\} \right), \quad H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i,$$

con canales locales de bajada/subida $L_{i,-} = \sqrt{\gamma(\bar{n}+1)}\sigma_i^-$, $L_{i,+} = \sqrt{\gamma\bar{n}}\sigma_i^+$ y $\bar{n} = \bar{n}(2h, T)$ la ocupación de Bose del baño. (H y $\{L_\mu\}$ los construye `build_ising`.)

Paso 1 — Problema de valor inicial. Fijada una preparación ρ_0 (un estado de Gibbs a T_0), la evolución es el PVI $\dot{\rho} = \mathcal{L}[\rho]$, $\rho(0) = \rho_0$, planteado *en el espacio de operadores*. No hay malla espacial que discretizar: la “malla” es la propia base de Hilbert ($d = 2^N$ y ρ es $d \times d$). Lo que se discretiza es el **tiempo**.

Paso 2 — Semidiscretización temporal (RK4). Se parte $[0, t_{\max}]$ en pasos Δt con $\rho_n \approx \rho(n\Delta t)$ y se aplica el **Runge–Kutta clásico de 4.º orden** (explícito, un paso):

$$\begin{aligned} k_1 &= \mathcal{L}[\rho_n], & k_2 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2}k_1], & k_3 &= \mathcal{L}[\rho_n + \frac{\Delta t}{2}k_2], \\ k_4 &= \mathcal{L}[\rho_n + \Delta t k_3], & \rho_{n+1} &= \rho_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

Error local $\mathcal{O}(\Delta t^5)$, global $\mathcal{O}(\Delta t^4)$.

Paso 3 — La operación discreta fundamental ($\mathcal{L}$ *matrix-free*). Cada k_i es una evaluación de $\mathcal{L}[\cdot]$. El punto clave de HPC es que **no** se forma el superoperador $\mathbb{L}$ de tamaño $d^2 \times d^2$; se evalúa la acción directamente sobre la matriz ρ con productos $d \times d$:

$$\mathcal{L}[V] = -i(HV - VH) + \sum_{\mu} \left(L_{\mu} V L_{\mu}^{\dagger} - \frac{1}{2}(L_{\mu}^{\dagger} L_{\mu} V + V L_{\mu}^{\dagger} L_{\mu}) \right).$$

Para la cadena de Ising son $2 + 3N_J$ productos de matrices densas $d \times d$ ($N_J = 2N$ canales), de coste $\mathcal{O}(d^3)$ cada uno; el coste por paso es $\mathcal{O}(N d^3)$. *Este producto de matrices es el hotspot que se paraleliza* (Listado 2).

1.3 Implementación serial y líneas de código

El núcleo está en `common/lindblad.hpp` (acción `apply_lindblad`, modelo `build_ising`, Gibbs por tiempo imaginario) y el integrador en `a_integracion_directa/src/rk4_evolution.cpp`. El Paso 2 (esquema RK4) es literalmente la función `rk4_step`:

```

1 static Mat rk4_step(const Mat& H, const std::vector<Mat>& Ls, const std::vector<Mat>& Lds,
2                     const std::vector<Mat>& LdL, const Mat& rho, double dt, int d) {
3     Mat k1 = apply_lindblad(H, Ls, Lds, LdL, rho, d);
4     Mat t2 = rho; axpy(t2, k1, 0.5*dt); Mat k2 = apply_lindblad(H, Ls, Lds, LdL, t2, d);
5     Mat t3 = rho; axpy(t3, k2, 0.5*dt); Mat k3 = apply_lindblad(H, Ls, Lds, LdL, t3, d);
6     Mat t4 = rho; axpy(t4, k3, dt); Mat k4 = apply_lindblad(H, Ls, Lds, LdL, t4, d);
7     Mat out = rho;
8     for (int i = 0; i < d*d; ++i) out[i] += dt*(k1[i] + 2.0*k2[i] + 2.0*k3[i] + k4[i])
9         /6.0;
10    return out;
}

```

Listing 1: Paso RK4 sobre la ecuación maestra (`rk4_evolution.cpp`). Las cuatro etapas $k_1 \dots k_4$ son cuatro llamadas a la acción matrix-free `apply_lindblad` (Paso 3).

1.4 La directiva de paralelización (OpenMP)

La preparación inicial es un Gibbs a $T_0 = 5$ y la referencia el Gibbs del baño a $T = 0,8$. El *hotspot* absoluto es el producto de matrices densas `matmul`, y ahí —y solo ahí— está la directiva OpenMP: una única `#pragma omp parallel for` reparte las filas i del producto entre los hilos (cada fila es independiente, sin condiciones de carrera):

```

1 inline Mat matmul(const Mat& A, const Mat& B, int d) {
2     Mat C(d * d, cd(0, 0));
3     #pragma omp parallel for schedule(static)    // <-- DIRECTIVA: reparte las filas i
4     // entre hilos
5     for (int i = 0; i < d; ++i) {
6         for (int k = 0; k < d; ++k) {
7             cd a = A[IDX(i, k, d)];
8             if (a == cd(0, 0)) continue;
9             const cd* brow = &B[IDX(k, 0, d)];
10            cd* crow = &C[IDX(i, 0, d)];
11            for (int j = 0; j < d; ++j) crow[j] += a * brow[j];
12        }
13    }
14    return C;
15 }

```

Listing 2: Producto de matrices con la directiva OpenMP (`common/lindblad.hpp`). Es el único punto paralelizado: el mismo binario, lanzado con distinto `OMP_NUM_THREADS`, da las curvas serie vs paralelo.

1.5 Justificación de la paralelización: OpenMP

El patrón de cómputo de la integración directa es: *miles de pasos secuenciales* (dependencia temporal estricta: ρ_{n+1} necesita ρ_n), cada uno dominado por productos de matrices densas que reutilizan los mismos datos. La dependencia temporal **impide** paralelizar entre pasos; el paralelismo disponible está *dentro* de cada paso, en el `matmul`. Ese es un patrón de **memoria compartida** con datos regulares y de tamaño moderado ($d \times d$ cabe en caché/RAM de un nodo): el paradigma natural es **OpenMP**, paralelizando el bucle de filas del `matmul` (`#pragma omp parallel for`). MPI sería contraproducente aquí (distribuir matrices pequeñas introduce comunicación de halo en cada uno de los miles de pasos); CUDA daría más ancho de banda pero no hay GPU disponible en la máquina de prueba (se discute como trabajo futuro en §2 y en la tabla ??). La corrección de la versión paralela se verifica de forma exacta: serial y multinúcleo producen el *mismo* estado final (coincidencia a precisión de máquina del observable D_{HS} final).

1.6 Comparación serial vs paralelo

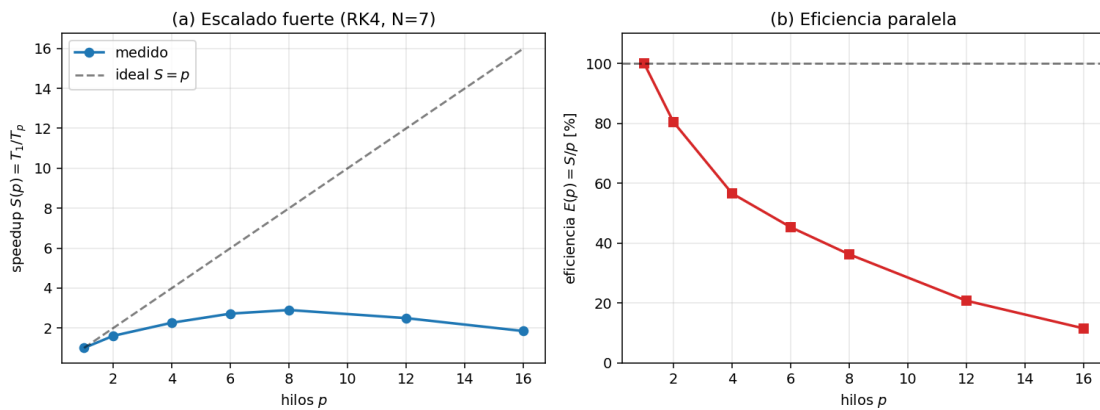


Figura 1: Escalado fuerte del integrador RK4 (cadena de Ising disipativa $N = 7$, $d = 128$) en un AMD Ryzen 7 7730U (8 núcleos físicos / 16 hilos). (a) speedup $S(p) = T_1/T_p$ frente al ideal $S = p$; (b) eficiencia paralela $E(p) = S/p$.

El speedup es **sublineal** y satura cerca del número de núcleos físicos: se alcanza $S_{\max} \approx 2,9$ con 8 hilos, con eficiencia del 36 % a 8 hilos. El ajuste a la **ley de Amdahl** $S(p) = 1/[f + (1 - f)/p]$ da una fracción serie efectiva $f \approx 0,25$. Las causas de la sublinealidad son típicas de este patrón: (i) el

integrador hace *fork/join* de OpenMP en cada uno de los muchos `matmul` pequeños, con sobrecoste acumulado; (ii) el problema es *memory-bandwidth bound* (las matrices se releen de memoria), de modo que los hilos compiten por el ancho de banda compartido; (iii) las asignaciones de memoria temporales y las operaciones vectoriales fuera del `matmul` son seriales. El salto de 8 a 16 hilos aporta poco porque los 16 hilos son *hyperthreads* sobre 8 núcleos físicos, sin ancho de banda adicional.

Para ver *directamente* la mejora —y no solo el speedup adimensional— la Figura 2 enfrenta el **tiempo de cómputo en serie (1 hilo) y en paralelo (8 hilos)** en función del tamaño del sistema N (la malla del operador, $d = 2^N$), a igual número de pasos. Ambas curvas crecen como $\sim d^3$, pero la paralela queda sistemáticamente por debajo: la *separación* entre las dos es la ganancia, y crece con la malla porque a mayor d el `matmul` domina y amortiza mejor el coste de *fork/join*.

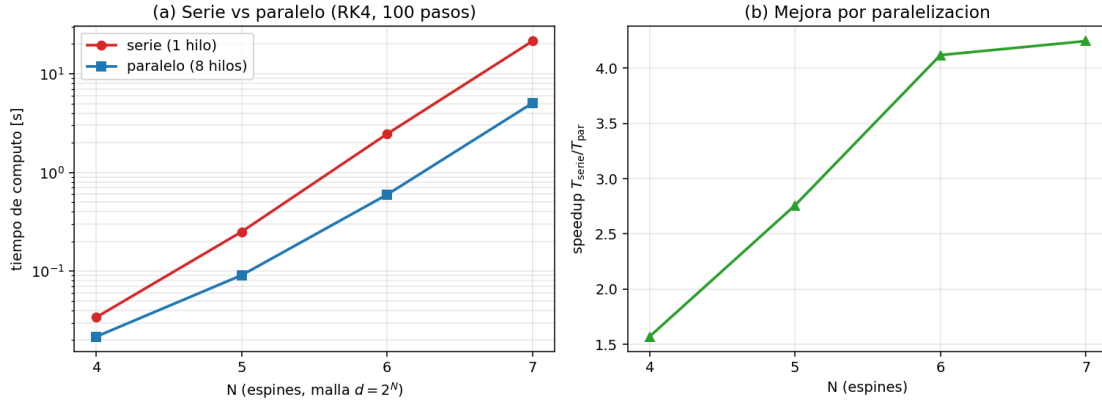


Figura 2: Tiempo de cómputo **serie vs paralelo** del RK4 frente al tamaño del sistema N (malla $d = 2^N$), a 100 pasos fijos. (a) las dos curvas (1 hilo en rojo, 8 hilos en azul, escala log) muestran el mismo coste $\sim d^3$ desplazado: el hueco entre ellas es la mejora. (b) el speedup $T_{\text{serie}}/T_{\text{par}}$ resultante crece con la malla (de $\sim 1,6$ en $N = 4$ a ~ 4 en $N = 7$): cuanto mayor es d , más domina el `matmul` y mejor se amortiza el *fork/join*, aunque siempre sublineal frente al ideal de 8.

1.7 Resultados y validación

Correctitud y orden del método. La figura 3 valida el integrador frente a la evolución espectral *exacta* $e^{t\mathcal{L}}\rho_0$ (suma de modos): el error a tiempo fijo decae como Δt^4 , confirmando el **orden 4** del RK4 (pendiente medida $\approx 4,04$).

Coste con el tamaño. La figura 4 confirma el escalado $\mathcal{O}(d^3)$ por paso esperado del producto de matrices densas: al pasar de N a $N + 1$ ($d \rightarrow 2d$) el coste por paso se multiplica por ~ 8 . Es justamente este crecimiento el que vuelve inviable la representación densa de ρ para N grande y motiva las trayectorias (§??) en el régimen de muchos cuerpos.

Conclusión del método (a). La integración directa por RK4 es *simple, exacta a orden 4 y robusta*, y se paraleliza de forma natural con OpenMP a nivel del producto de matrices. Su limitación es doble: el coste $\mathcal{O}(d^3)$ por paso y la necesidad de pasos pequeños por estabilidad/precisión, que obligan a muchas evaluaciones del Liouvilliano. El método (b) (acción del exponencial, §2) ataca precisamente este segundo punto.

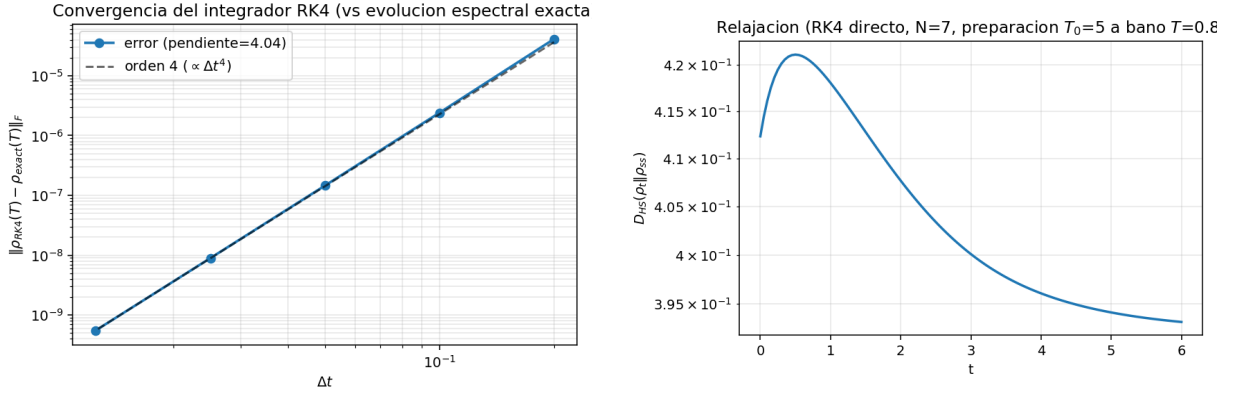


Figura 3: Izquierda: convergencia de orden 4 del RK4 frente a la solución exacta. Derecha: curva de relajación $D_{HS}(t)$ producida por el método (preparación caliente $T_0 = 5$ relajando al baño $T = 0,8$).

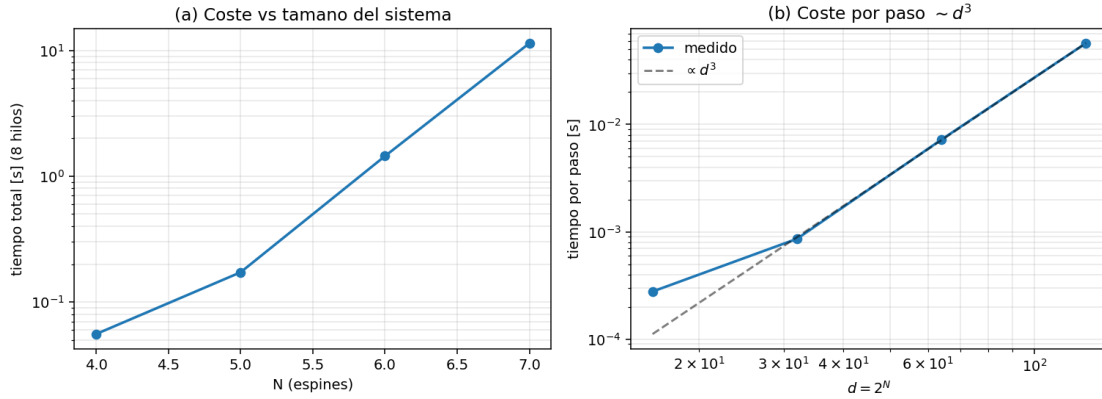


Figura 4: Coste del método con el tamaño del sistema (8 hilos). (a) tiempo total; (b) tiempo por paso frente a la referencia $\propto d^3$.

2 Evolución temporal (b): acción del exponencial por Krylov

2.1 Explicación del framework

El método (b) resuelve la misma evolución $\dot{\rho} = \mathcal{L}[\rho]$ pero por una vía distinta: en lugar de integrar paso a paso, aproxima directamente la **acción del exponencial del Liouvilliano**,

$$\rho(t + \tau) = e^{\tau \mathcal{L}}[\rho(t)],$$

que es la solución *exacta* en un paso de tamaño τ . La dificultad es que $e^{\tau \mathcal{L}}$ es un objeto de dimensión $d^2 \times d^2$ imposible de formar. La idea de Krylov es proyectar la acción sobre el subespacio $\mathcal{K}_m = \text{span}\{\rho, \mathcal{L}\rho, \dots, \mathcal{L}^{m-1}\rho\}$, de dimensión $m \ll d^2$, donde el exponencial es trivial de calcular.

2.2 Del modelo teórico a la discretización (paso a paso)

Paso 0 — Modelo teórico. El mismo de (a): la ecuación maestra GKSL $\dot{\rho} = \mathcal{L}[\rho]$ para la cadena de Ising disipativa (Hamiltoniano H y canales L_μ idénticos, §1.2).

Paso 1 — Solución formal exacta. En lugar de integrar la EDO, se usa que la solución del PVI es *exacta* en un paso de tamaño τ mediante el exponencial del generador:

$$\rho(t + \tau) = e^{\tau \mathcal{L}}[\rho(t)].$$

El obstáculo: $e^{\tau\mathcal{L}}$ vive en el superoperador de dimensión $d^2 \times d^2$, imposible de formar y exponenciar para N grande.

Paso 2 — Discretización por proyección de Krylov (Arnoldi). Se proyecta la acción del exponencial sobre el subespacio de Krylov $\mathcal{K}_m = \text{span}\{\rho, \mathcal{L}\rho, \dots, \mathcal{L}^{m-1}\rho\}$ ($m \ll d^2$). Con **Arnoldi** se construye una base ortonormal $\{Q_0, \dots, Q_{m-1}\}$ (cada Q_i es un operador $d \times d$; producto interno de Frobenius $\langle A, B \rangle = \text{Tr}(A^\dagger B)$) y la matriz de Hessenberg $H_m \in \mathbb{C}^{m \times m}$ que representa $\mathcal{L}$ en esa base. El *único* ingrediente que toca el sistema grande es la acción $w = \mathcal{L}[Q_j]$ en cada iteración de Arnoldi.

Paso 3 — Exponencial pequeño y reconstrucción. La acción discretizada es

$$e^{\tau\mathcal{L}}[\rho] \approx \|\rho\|_F \sum_{i=0}^{m-1} [e^{\tau H_m}]_{i,0} Q_i,$$

con $e^{\tau H_m}$ calculado sobre la matriz *pequeña* $m \times m$ por *scaling-and-squaring* con serie de Taylor (`small_expm`). Coste por bloque: m aplicaciones de $\mathcal{L}$ (Arnoldi) + $\mathcal{O}(m d^2)$ y un $e^{\tau H_m}$ de coste $\mathcal{O}(m^3)$ despreciable. La ventaja decisiva: para τ dentro del radio de convergencia, $m \sim 20\text{--}30$ basta para precisión de máquina y *un solo bloque avanza un paso de tiempo grande*.

2.3 Implementación serial y líneas de código

En `b_accion_exponencial/src/krylov_evolution.cpp`. El bucle de Arnoldi (Paso 2) es el corazón del método; nótese que cada iteración llama a la misma acción `apply_lindblad` de (a) —es decir, *comparte el hotspot paralelizable*—:

```

1 for (int j = 0; j < m; ++j) {
2     Mat w = apply_lindblad(H, Ls, Lds, LdL, Q[j], d); ++L_applies;    // <-- hotspot (
3     // ortogonalizacion de Gram-Schmidt
4     for (int i = 0; i <= j; ++i) {
5         cd hij = fro_dot(Q[i], w); Hess[i*m+j] = hij; axpy(w, Q[i], -hij);
6     }
7     double hj = fro_norm(w);
8     Hess[(j+1)*m+j] = hj;
9     if (hj < 1e-12) { mm = j + 1; break; }
10    Q.push_back(scaled(w, 1.0 / hj));
11 }

```

Listing 3: Bucle de Arnoldi de `krylov_step` (`krylov_evolution.cpp`): la línea `apply_lindblad(...)` es la acción matrix-free de $\mathcal{L}$, donde se cuenta el trabajo (`L_applies`) y donde reside la paralelización OpenMP (vía `matmul`, Listado 4).

2.4 La directiva de paralelización (OpenMP)

El *hotspot* es el mismo que en (a): el producto de matrices densas, llamado m veces por bloque dentro de `apply_lindblad`. La directiva es por tanto idéntica —una sola `#pragma omp parallel for` en `matmul` (`common/lindblad.hpp`)—, lo que explica que el escalado paralelo de (b) sea equivalente al de (a):

```

1 #pragma omp parallel for schedule(static)    // <-- DIRECTIVA: filas i repartidas entre
2     hilos
3 for (int i = 0; i < d; ++i)
4     for (int k = 0; k < d; ++k) { /* C[i,:] += A[i,k] * B[k,:] */ }

```

Listing 4: La misma directiva OpenMP que en T2(a): el bucle de filas de `matmul` es el único punto paralelizado (compartido por RK4 y Krylov).

2.5 Justificación de la paralelización: OpenMP

Como en el método (a), el *hotspot* es la acción $\mathcal{L}[\cdot]$ (el producto de matrices densas), llamada m veces por bloque. La ortogonalización de Arnoldi añade productos internos de Frobenius de coste $\mathcal{O}(d^2)$, mucho menores que el $\mathcal{O}(d^3)$ del `matmul`. Por tanto el patrón sigue siendo de **memoria compartida** dominado por `matmul`, y el paradigma natural vuelve a ser **OpenMP** sobre el mismo bucle. (En el régimen de muchos cuerpos, donde d^2 no cabe en memoria, la acción de Krylov se paraleliza con MPI distribuyendo el vector $|\rho\rangle$; aquí, con ρ denso en un nodo, OpenMP es lo adecuado.) La corrección se verifica de dos formas: serial y paralelo dan el mismo resultado, y el resultado coincide con el del método (a) (RK4) a todas las cifras: $D_{HS}^{\text{final}} = 0,4241128226$ para $N = 6$, idéntico por ambos métodos (referencia exacta por evolución espectral: $D_{HS}^{\text{exacto}} = 0,42411283$).

2.6 Comparación serial vs paralelo

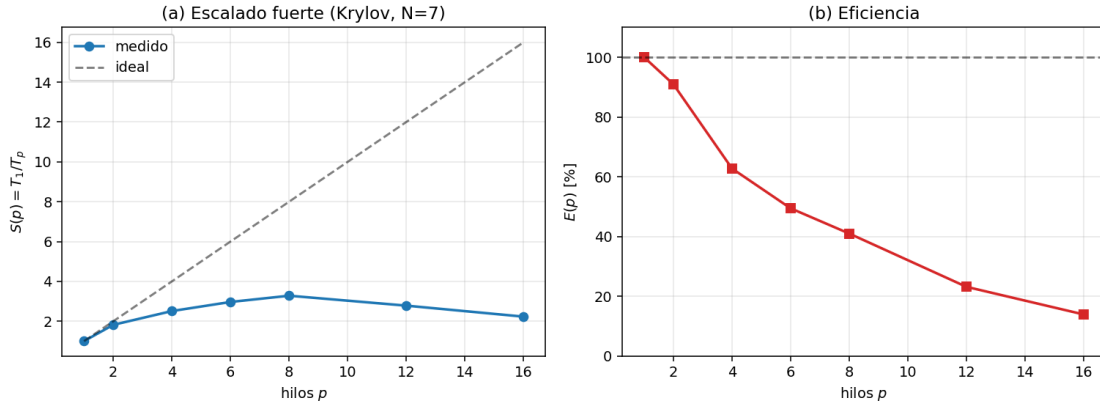


Figura 5: Escalado fuerte del método de Krylov (Ising disipativo $N = 7$, $d = 128$). El comportamiento es análogo al del método (a): speedup sublineal que satura en los núcleos físicos, por las mismas causas (fork/join del `matmul`, *memory-bandwidth bound*, hyperthreading sin ancho de banda extra). Pico $S_{\text{max}} \approx 3,3$, eficiencia 41 % a 8 hilos (ligeramente mejor que RK4: hay menos llamadas a `matmul` por unidad de trabajo).

La Figura 6 muestra la mejora de forma directa: el **tiempo de cómputo en serie (1 hilo) frente al paralelo (8 hilos)** en función del tamaño del sistema N (malla $d = 2^N$), a igual paso τ y misma dimensión de Krylov m . Igual que en (a), las dos curvas crecen como $\sim d^3$ y el hueco entre ellas —la ganancia— se ensancha con la malla.

2.7 Comparación de método (a vs b): el resultado central

La ventaja de Krylov no está en el escalado paralelo (idéntico al de RK4) sino en el **trabajo necesario para una precisión dada**. La figura 7 mide el error del observable D_{HS} final frente a una referencia de alta precisión, en función del número de aplicaciones de $\mathcal{L}$ (la operación cara, independiente de la máquina):

La interpretación física/numérica: RK4 converge *algebraicamente* (el error baja como Δt^4), mientras que Krylov converge *super-algebraicamente* (casi exponencial en m) porque captura la acción exacta del exponencial dentro del subespacio. En este sistema disipativo *no rígido*, ambos métodos son de trabajo comparable para precisiones moderadas, pero Krylov presenta tres ventajas estructurales: (i) a igual trabajo es $\sim 25\times$ más preciso; (ii) usa pasos de tiempo $5\times$ mayores ($\tau = 1,0$ frente a $\Delta t = 0,2$) manteniéndose estable y exacto; (iii) es **incondicionalmente estable**, mientras que RK4 tiene un límite de estabilidad $\Delta t < 2,8/|\lambda_{\text{max}}|$. Esta ventaja se vuelve *decisiva* para generadores *rígidos* (escalas de energía grandes o disipación fuerte) y para alcanzar el estado estacionario, donde el límite de estabilidad obliga a RK4 a dar muchísimos pasos diminutos. RK4

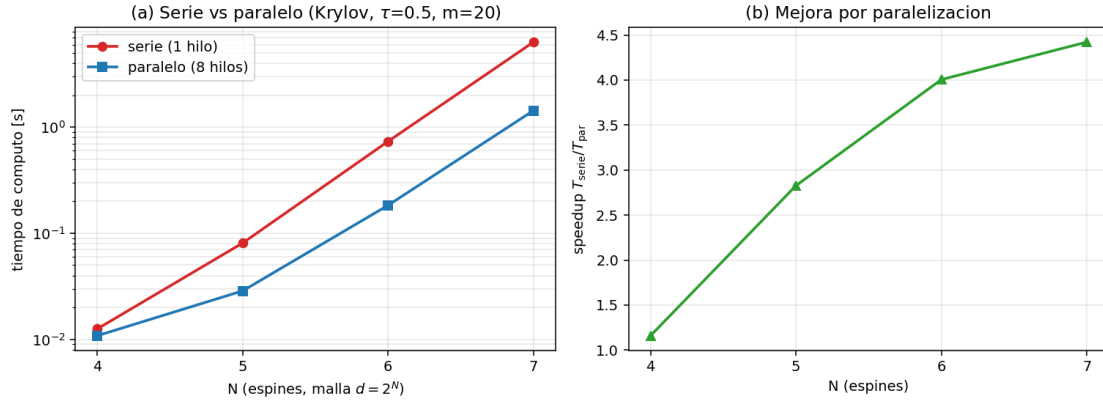


Figura 6: Tiempo de cómputo **serie vs paralelo** del método de Krylov frente al tamaño del sistema N (malla $d = 2^N$), a $\tau = 0,5$ y $m = 20$ fijos. (a) curvas de 1 hilo (rojo) y 8 hilos (azul) en escala log; (b) speedup $T_{\text{serie}}/T_{\text{par}}$ resultante. El patrón es el mismo que el de RK4 porque el hotspot paralelo —el `matmul` de la acción de $\mathcal{L}$ — es común a ambos métodos.

sigue siendo preferible cuando se necesita la trayectoria *densamente* muestreada en el tiempo o cuando el Liouvilliano depende del tiempo.

2.8 Resultados y conclusión del método (b)

La figura 8 muestra la curva de relajación obtenida con pasos *grandes* ($\tau = 0,25$), inalcanzables para RK4 a esa precisión. Resumen de la comparación a–b:

- Misma física (resultados idénticos), misma paralelización (OpenMP sobre `matmul`, escalado sublineal equivalente).
- Krylov (b): mucho menos trabajo para alta precisión y pasos de tiempo grandes; ideal para llegar al estacionario o muestrear pocos tiempos.
- RK4 (a): más simple, robusto, y mejor para trayectorias densas o generadores dependientes del tiempo.

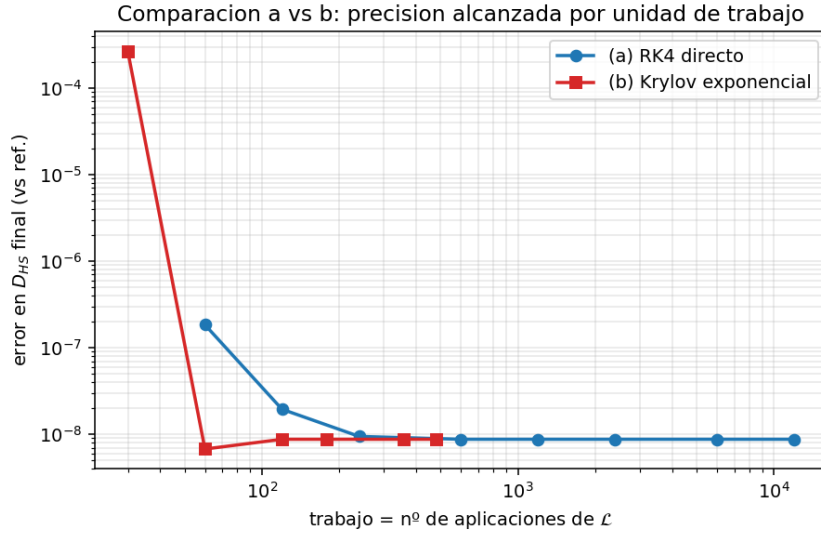


Figura 7: Error del observable D_{HS} final frente a la referencia exacta (evolución espectral), en función del trabajo. **A igual trabajo (≈ 60 aplicaciones de $\mathcal{L}$), Krylov es $\sim 25\times$ más preciso** (7×10^{-9} frente a 2×10^{-7} de RK4). El punto Krylov de $m = 10$ (no convergido, 2×10^{-4}) cae cinco órdenes de magnitud al pasar a $m = 20$: convergencia *super-algebraica* en m , frente a la algebraica ($\propto \Delta t^4$) de RK4. El suelo $\sim 9 \times 10^{-9}$ es la precisión de la propia referencia.

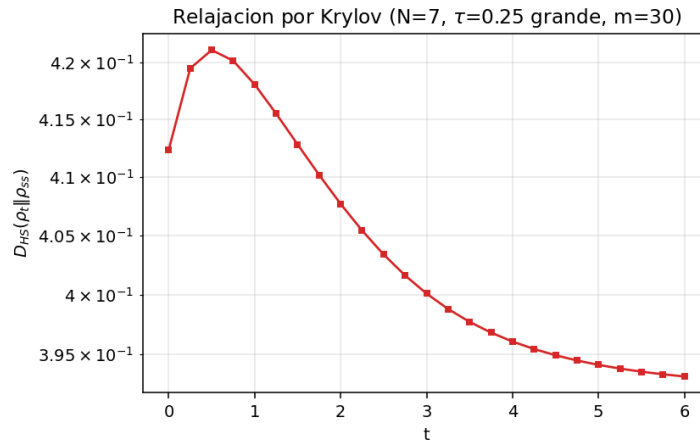


Figura 8: Relajación $D_{HS}(t)$ por Krylov con pasos grandes $\tau = 0,25$ ($N = 7$).